

🎵 Music Streaming Analytics System

A Spotify-style music streaming analytics platform built for **DBMS Project (UCS310)**
Thapar Institute of Engineering and Technology

📌 Project Overview

A complete database-driven analytics system that simulates a music streaming platform. The system stores users, artists, songs, subscription plans, and streaming events — then exposes analytical insights through SQL queries, views, stored procedures, and a live web dashboard.

📁 Project Structure

```
MusicStreamingAnalytics/
|
├── backend/                ← FastAPI Python backend
|   ├── main.py             ← All API routes (13 endpoints)
|   ├── database.py         ← SQLite setup + sample data
|   ├── requirements.txt    ← Python dependencies
|   └── static/
|       └── index.html      ← Live frontend dashboard (connected to backend)
|
├── database/
|   └── music_streaming.sql  ← Complete SQL file (MySQL 5.7 compatible)
|                               DDL · DML · Views · Procedures · Functions
|                               Triggers · Cursors · Transactions
|
├── dashboard/
|   └── standalone_dashboard.html ← Standalone frontend (no backend needed)
|                                   Open directly in browser
|
├── docs/
|   └── synopsis.md          ← Project synopsis and documentation
|
└── README.md               ← This file
```

Database Design

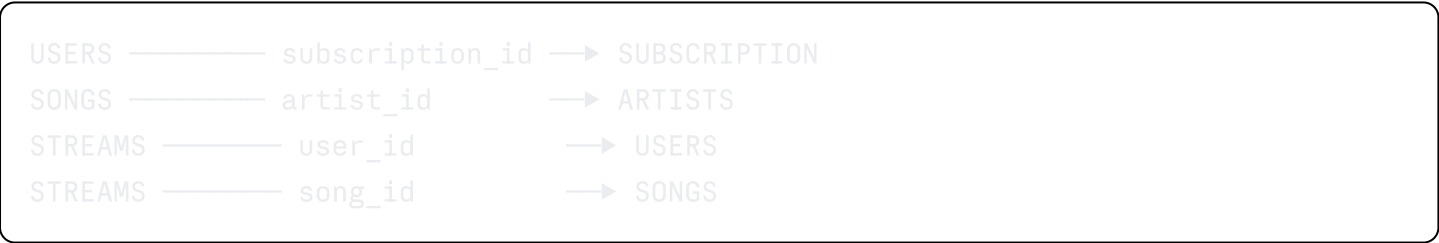
Tables (5)

Table	Description	Rows
SUBSCRIPTION	Plan types (Free, Premium, Family)	3
ARTISTS	Artist profiles	10
SONGS	Song catalog	23
USERS	Registered users	15
STREAMS	Every play event (fact table)	51

Normalization

- **1NF** — Atomic values, no repeating groups
- **2NF** — No partial dependencies
- **3NF** — No transitive dependencies (plan price lives in SUBSCRIPTION, not USERS)

Relationships



SQL Features Implemented

Category	Items
DDL	CREATE TABLE, ALTER TABLE, DROP, Indexes, Constraints
DML	INSERT (103 rows), UPDATE, DELETE
Views	popular_songs, user_listening_history, revenue_dashboard, artist_performance
Analytical Queries	12 queries — JOINS, subqueries, correlated subqueries, GROUP BY, HAVING, aggregates
Stored Procedures	add_stream, update_subscription, generate_monthly_report

Category	Items
Functions	calculate_user_listening_time, get_artist_popularity_score, is_premium_user
Triggers	before_stream_insert (validation), after_subscription_change (audit log)
Cursors	show_top_artists (explicit FETCH loop)
Transactions	COMMIT, ROLLBACK, SAVEPOINT

 Option 1 — Run with Backend (Full Stack)

Requirements

- Python 3.8+

Steps

```
bash

# 1. Go into the backend folder
cd backend

# 2. Install dependencies
pip install fastapi uvicorn

# 3. Start the server
python main.py

# 4. Open browser
# http://127.0.0.1:8000
```

API Endpoints

Method	Endpoint	Description
GET	<code>/api/stats/overview</code>	KPI numbers
GET	<code>/api/stats/devices</code>	Streams by device
GET	<code>/api/stats/genres</code>	Genre breakdown
GET	<code>/api/stats/revenue</code>	Revenue per plan
GET	<code>/api/stats/peak-hours</code>	Listening by hour

Method	Endpoint	Description
GET	<code>/api/stats/locations</code>	Top locations
GET	<code>/api/songs</code>	Songs with <code>?search=&genre=&year=</code>
GET	<code>/api/songs/{id}</code>	Single song detail
GET	<code>/api/artists</code>	Artists with <code>?search=&verified=true</code>
GET	<code>/api/users</code>	Users with <code>?search=&plan=</code>
GET	<code>/api/users/{id}/history</code>	User listening history
POST	<code>/api/streams</code>	Log a new stream event
GET	<code>/api/search?q=</code>	Global search

Interactive API docs: `http://127.0.0.1:8000/docs`

🌐 Option 2 — Standalone Dashboard (No Installation)

Just open `dashboard/standalone_dashboard.html` in any browser.

All data is embedded — no server or database needed.

Good for quick demo or offline presentation.

📄 Option 3 — Run SQL on MySQL / OneCompiler

1. Go to onecompiler.com/mysql
2. Paste the contents of `database/music_streaming.sql`
3. Click **RUN**

Note: The `DROP DATABASE` and `CREATE DATABASE` lines at the top are commented out for online compilers. Uncomment them for local MySQL Workbench.

🔧 Tech Stack

Layer	Technology
Database	SQLite (backend) / MySQL 5.7 (SQL file)
Backend	Python · FastAPI · Uvicorn

Layer	Technology
Frontend	HTML · CSS · JavaScript · Chart.js
SQL	Standard SQL + MySQL PL/SQL

Team

Course: UCS310 — Database Management Systems
Institute: Thapar Institute of Engineering and Technology

License

This project is submitted for academic purposes under UCS310 DBMS coursework.